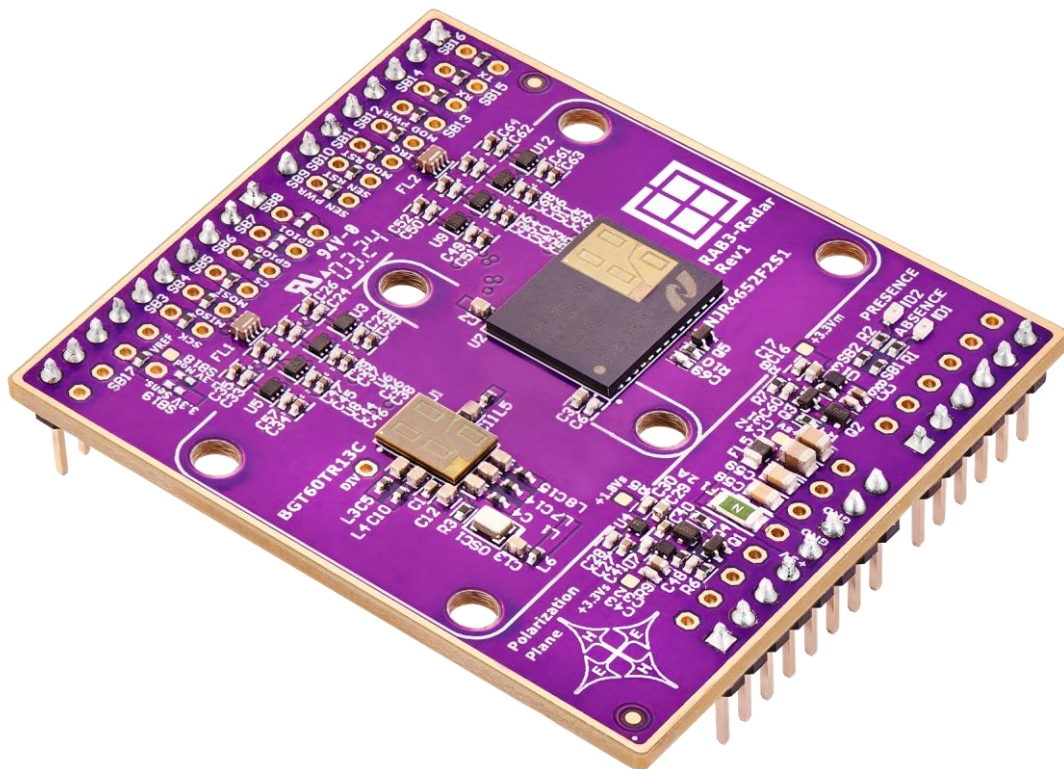


Presence and position detection with radar sensor

Application note



Versions

Version	Date	Rationale
1.0	May 19, 2026	First release. Author: ROJ, KOA

Legal disclaimer

The evaluation board is for testing purposes only and, because it has limited functions and limited resilience, is not suitable for permanent use under real conditions. If the evaluation board is nevertheless used under real conditions, this is done at one's responsibility; any liability of Rutronik is insofar excluded.

Table of contents

Table of contents	2
Introduction	3
Overview	3
Useful links	4
Hardware	4
Software step-by-step	5
Import project in Modus Toolbox	5
Copy keys, packets and policy	6
Build and program	7
Calculations	8
Raw values from radar sensor	8
Algorithm	9
To be implemented	9

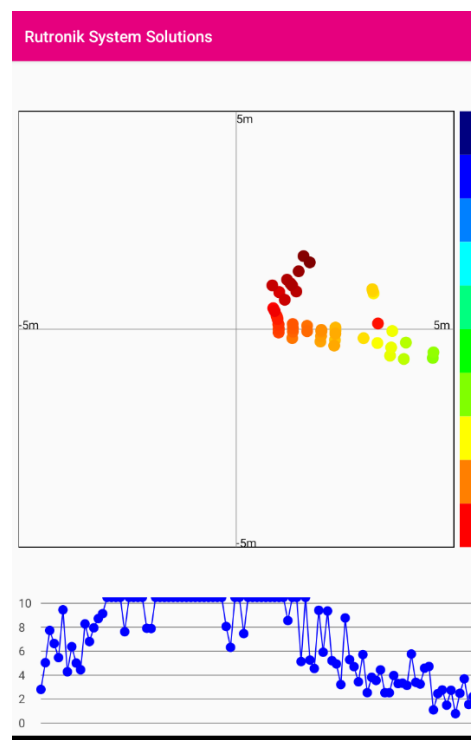
Introduction

This document describes the calculations behind the presence and position detection using Infineon BGT60TR13C radar sensor. Compared to the [earlier project](#) on this topic, we now use Rutronik's proprietary algorithm to detect the presence of an object and calculate its position.

Overview

The project goal is to detect if the object is present in front of the radar, and in case of presence, to detect the object position.

The top plot shows the position of a presented object. Red points indicate more recent detection, while blue points are the older one (presence has been detected around 20 seconds before). After 20 seconds, a detected point disappears.



The bottom plot shows an indicator used to detect if a presence has been detected or not. If the value is above 5, the algorithm considers that a presence has been detected.

This is a first draft of the application (only Android now). It will be enhanced soon.

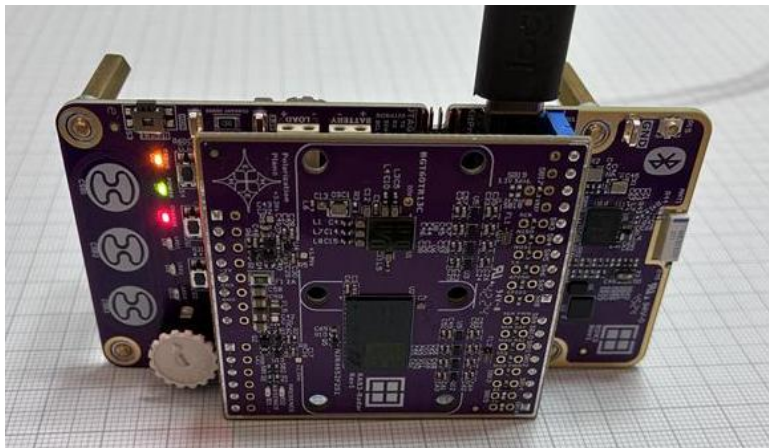
Useful links

1. GitHub project with the source code (firmware+GUI):
https://github.com/RutronikSystemSolutions/RDK3_Radar_Presence_Detection
2. Application note for the earlier version of the project
https://github.com/RutronikSystemSolutions/RAB3-Radar_Documents/blob/main/RAB3-Radar_AppNote_Presence_Detection.pdf

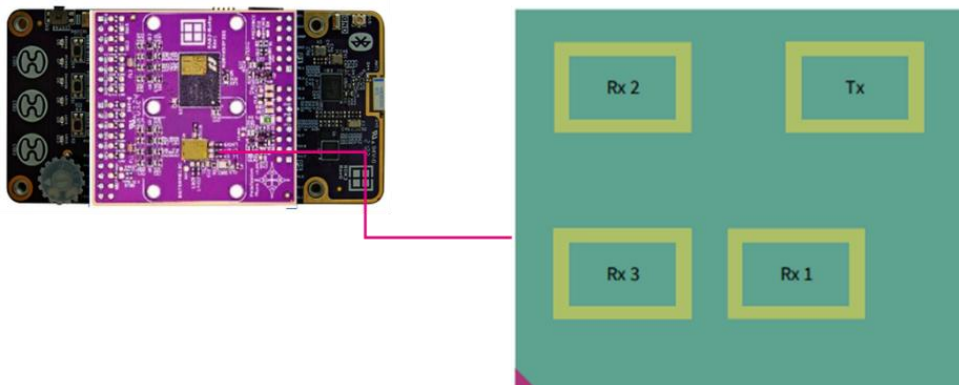
Hardware

- [RDK3](#)
- [RAB3-Radar](#)

It is important to place the RDK3 like this, otherwise the angle detection will not work.



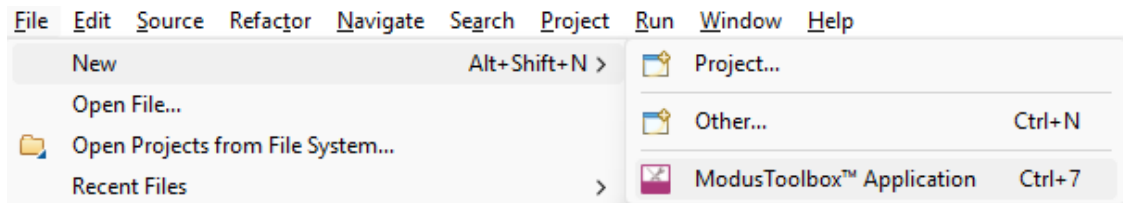
The algorithm uses information from RX3 and RX1 to determine the position of the target, so the orientation of the radar board matters.



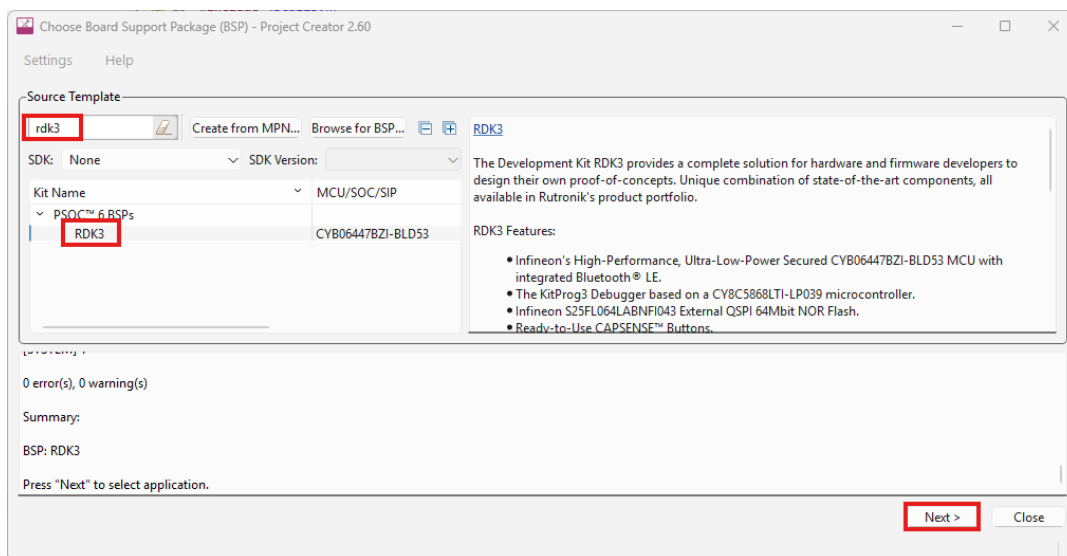
Software step-by-step

Import project in Modus Toolbox

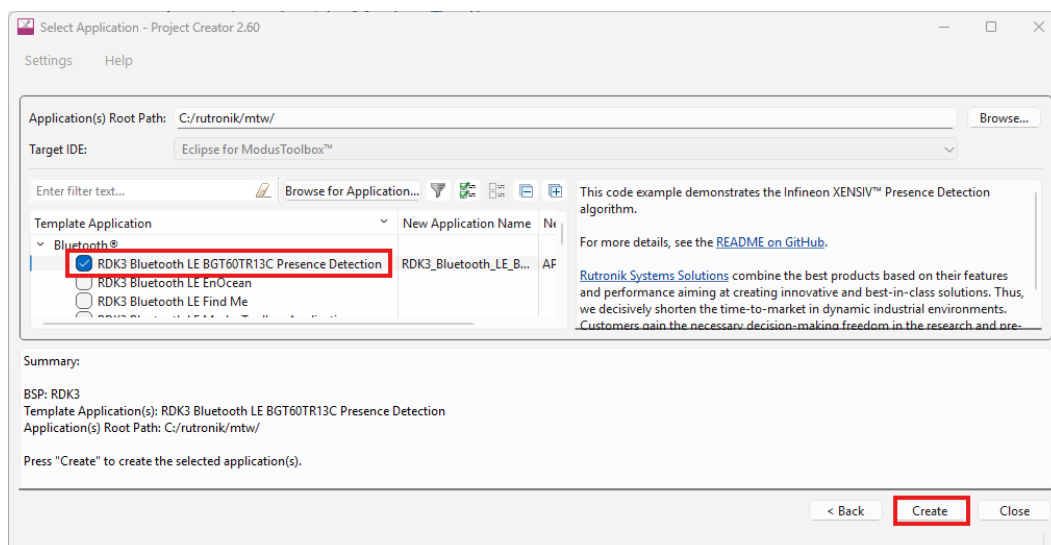
In **Eclipse for Modus Toolbox**, click on **File - New - ModusToolbox™ Application**.



In the search text area, type **RDK3**, select it in the list and click on **Next**.



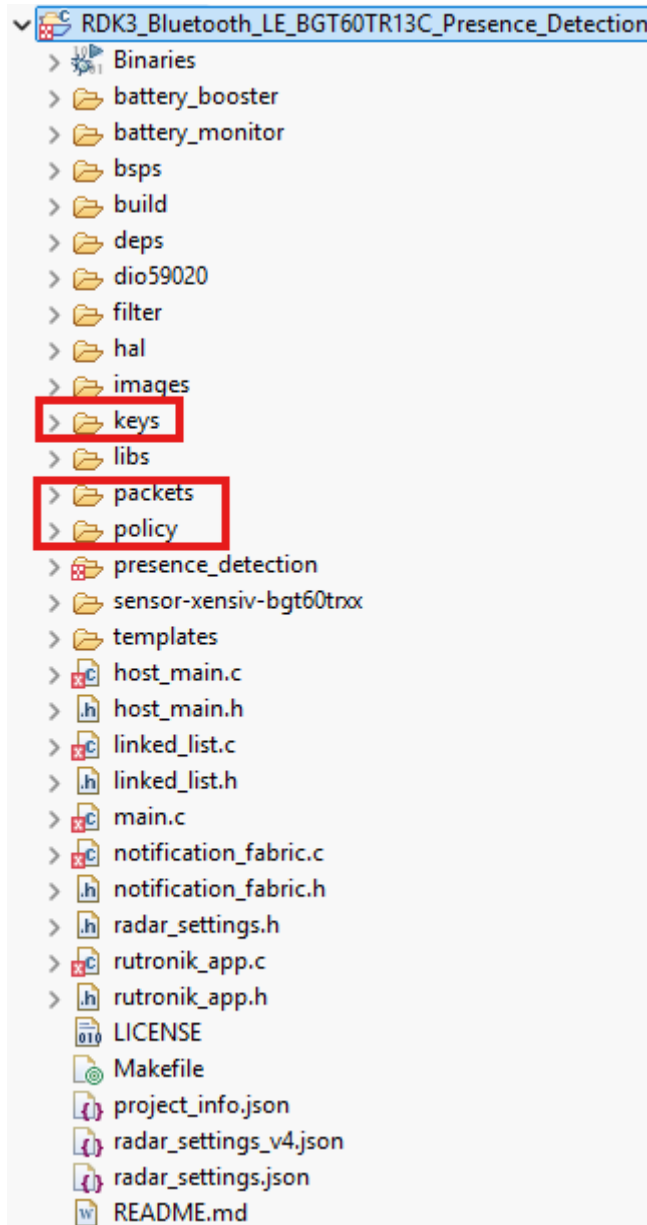
Expand the **Bluetooth®** category, select **RDK3 Bluetooth LE BGT60TR13C Presence Detection** and click on **Create**.



This will create a new project, that you will see in the project explorer.

Copy keys, packets and policy

Assumption: your RDK3 has already been provisioned.

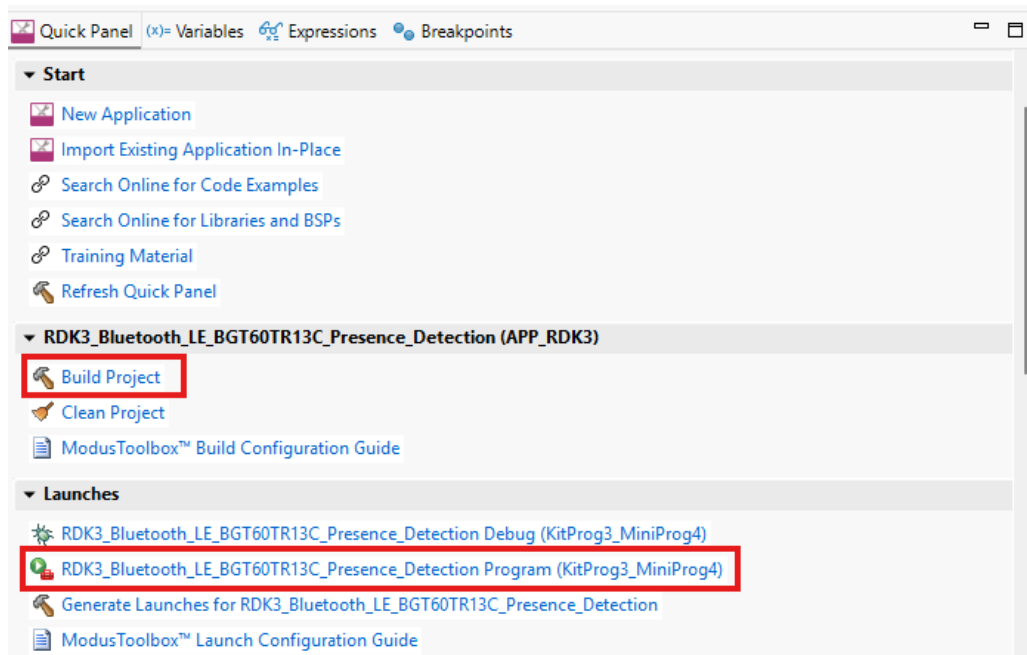


Copy your generated **keys**, **packets** and **policy** files directly inside the project.

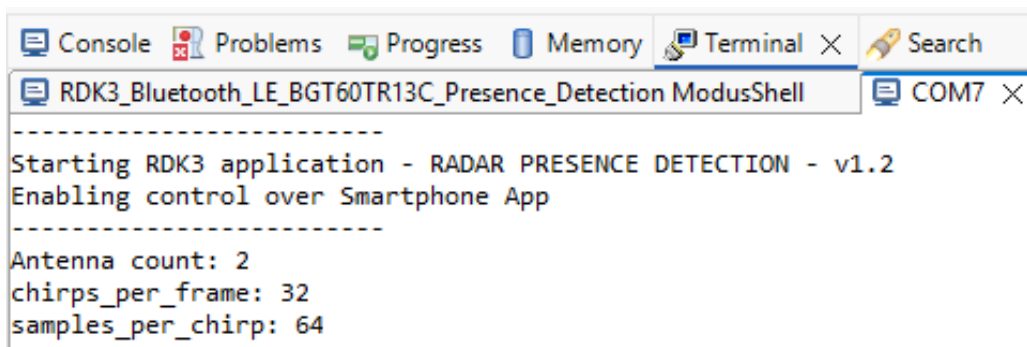
If your RDK3 is provisioned, but you do not have these files, you can find files that might be compatible with your RDK3 on our [GitHub](#).

Build and program

Click on **Build project** and then on **Program** to flash the device.



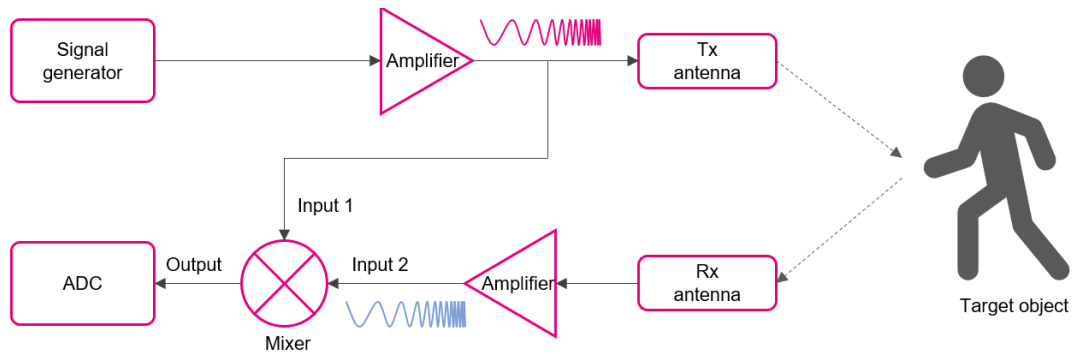
If you open a terminal, you will see something like that:



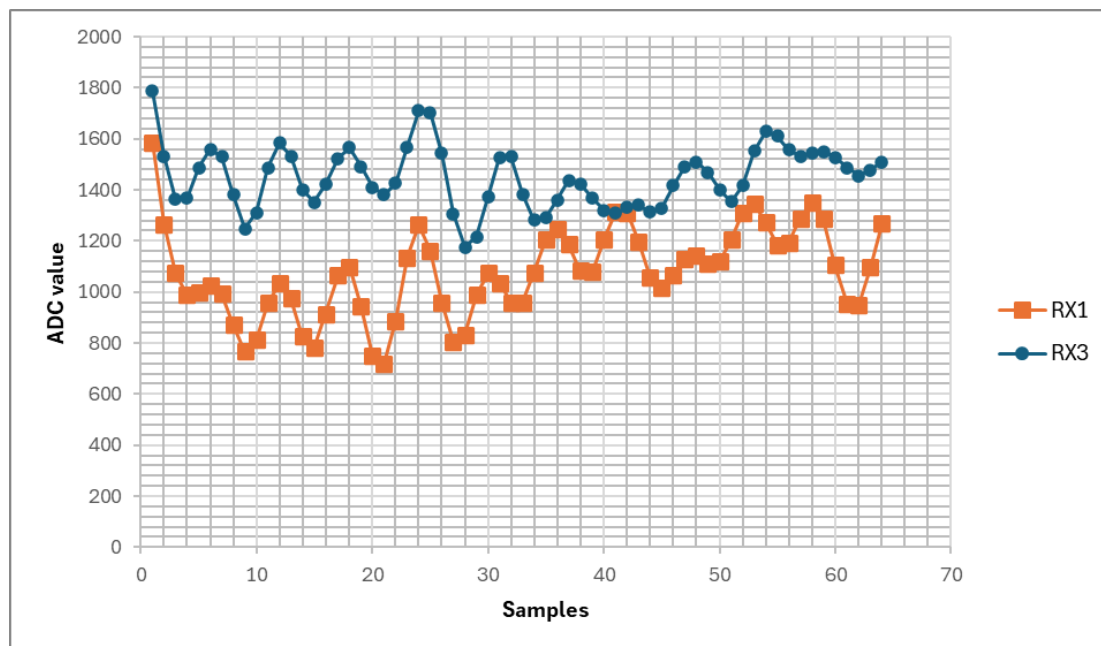
Calculations

Raw values from radar sensor

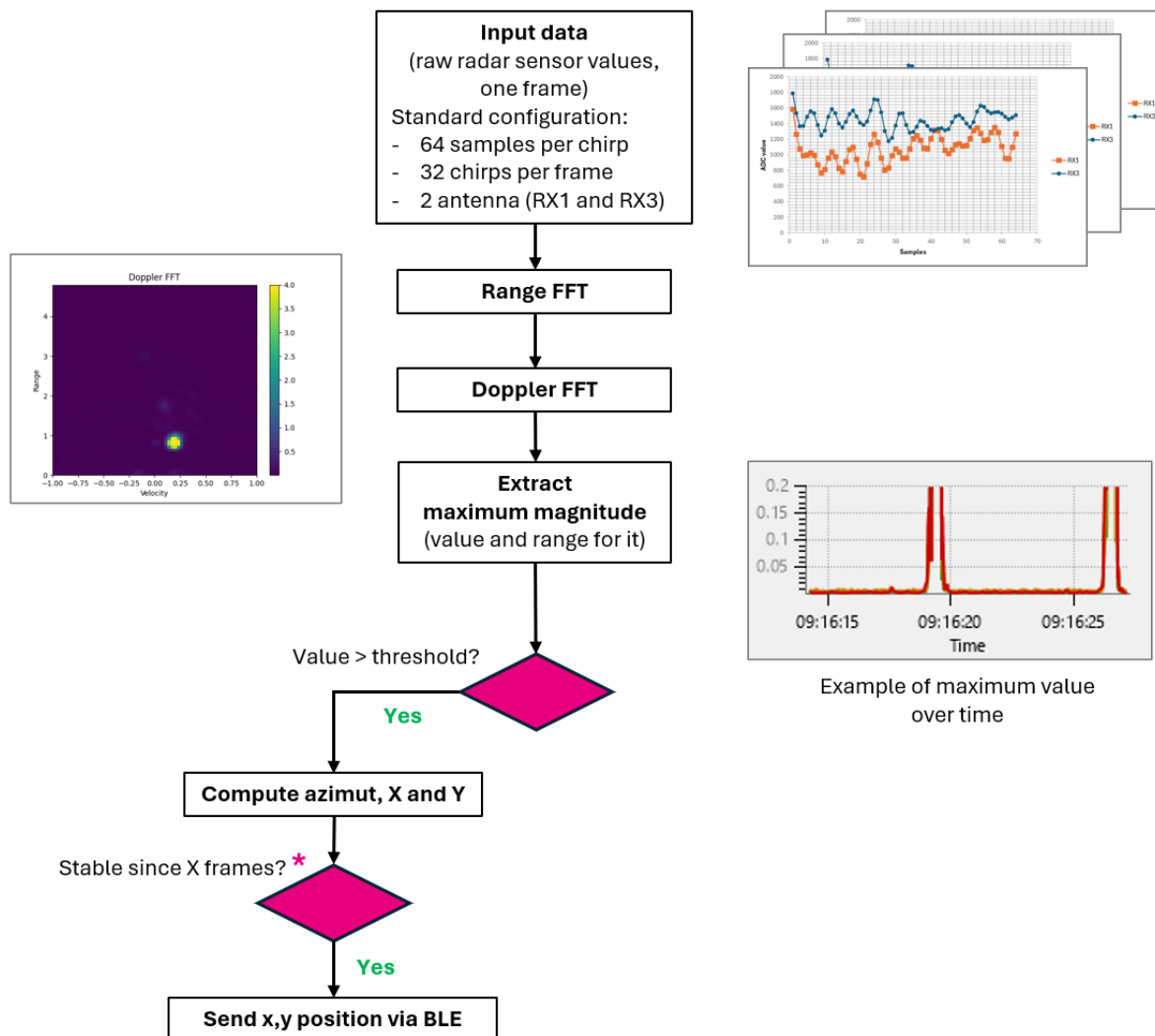
The BGT60TR13C delivers raw values (sampled by the internal ADC) over SPI bus.



One packet of raw values, also named frame, is composed by multiple chirps. Each chirp contains X (in our case 64) samples per antenna (as below).



Algorithm



* The algorithm keeps track of the computed x and y. If within X frames (frames are generated periodically – 10Hz in our case), the position (x,y) remains stable (within 0.5m distance in this example), then the point is considered as valid.

This enables to filter out some “ghost” targets.

To be implemented

- Use 2D-CFAR or DBSCAN on “Doppler Map”
- Use of CAPON to compute angle of arrival
- Instead of removing average before computing Doppler FFT, try to compute Doppler FFT and then to apply MTI using filter
- Use phase information (after range FFT) to determine micro motion or not.